MakeMyMarrige

DETAILED SYSTEM DESIGN & ARCHITECTURE

Version 2.0 | 3 November 2026

Aligned with Updated PRD v2.0

Kanpur, Uttar Pradesh, India

1. Document control

Field	Value
Product	MakeMyMarrige
Document	Detailed System Design & Architecture
Version	2.0
Date	3 November 2026
Status	Architecture baseline aligned to PRD v2.0; provider and deployment choices remain open where identified
Product baseline	MakeMyMarrige Updated Detailed PRD v2.0
Launch market	Kanpur, Uttar Pradesh, India
Primary stack	Next.js + TypeScript; NestJS + TypeScript; MongoDB Atlas; managed Redis/BullMQ; private Cloudflare R2
Supersedes	System Design v1.2 and Detailed System Design Architecture v1.3

This document translates the approved product requirements and workflows WF01–WF12 into an implementation-oriented architecture. It defines system boundaries, domain ownership, request processing, data design, transaction patterns, security controls, background processing, deployment, testing and operational procedures. Provider names and operational settings that have not been approved are explicitly marked as open decisions.

2. Executive summary

MakeMyMarrige is designed as a modular monolith: a Next.js web/PWA frontend, a separate NestJS REST API, and a separately deployed worker process using the same backend codebase. MongoDB Atlas is the authoritative system of record. Managed Redis provides BullMQ transport, distributed rate-limit counters and short-lived coordination; Redis is not the source of truth. Cloudflare provides edge protection and private Cloudflare R2 stores photo objects. External providers are accessed through adapters.

The pilot uses shared-database logical multi-tenancy. One wedding workspace is one tenant. Tenant-owned documents are scoped by weddingId. NestJS resolves the tenant from authenticated membership, never from a client-supplied tenant identifier. A user may have at most one ACTIVE or SUSPENDED wedding membership at a time; historical membership periods and later rejoining are allowed.

- Optimise for correctness, privacy, recoverability and maintainability before scale complexity.
- Use transactions for cross-document invariants and expectedVersion for optimistic concurrency.
- Use a MongoDB transactional outbox for durable asynchronous work and a webhook inbox for external callbacks.
- Treat provider callbacks as untrusted until authenticated and reconciled.
- Keep all tenant data, files, queues, caches, exports and worker operations tenant-scoped.
- Do not introduce microservices, Kubernetes, Kafka, sharding or event sourcing for the pilot.

3. Architecture principles and constraints

- Modular monolith with explicit bounded contexts and dependency direction.
- Separate API and worker processes, sharing domain/application modules and configuration.
- MongoDB is authoritative for business state; Redis is ephemeral infrastructure.
- Default deny: authentication, membership, permission and resource checks occur server-side.
- Every tenant-owned read/write/aggregation/export is scoped to weddingId.
- Explicit command endpoints perform state transitions; clients cannot directly patch privileged status fields.
- Money is integer paise; currency is INR; timestamps are UTC instants and wedding-local date/time rules use Asia/Kolkata.
- At-least-once job delivery is assumed; all handlers must be idempotent.
- External provider adapters isolate vendor-specific SDKs and webhook formats.
- No provider integration may silently imply success, availability, delivery or deletion without authoritative evidence.

4. System context and actors

Actor/system	Purpose	Trust boundary
Co-owner	Workspace governance, approvals, full permitted tenant access	Authenticated member; highest tenant role
Organiser	Operational work within explicit action permissions and scopes	Authenticated member; least privilege
Treasurer	Permitted financial operations and normal expense approval	Authenticated special membership role

Actor/system	Purpose	Trust boundary
Guest	RSVP, permitted photo upload/view and own Shagun history	Wedding-scoped OTP principal
Platform support	Exceptional support and recovery	Separate platform identity; break-glass only
Google Places	Vendor discovery provider	External API; terms/attribution apply
Email/OTP provider	Transactional email and guest OTP delivery	External provider; credentials isolated
UPI payment provider	Payment initiation, status and webhook events	External provider; verified server-side evidence
Cloudflare R2	Private photo object storage	External storage; access mediated by API

A guest is not automatically a platform user. A guest identity is wedding-specific. A phone number may be shared by multiple guest records; OTP proves control of the phone at that time, not the identity of every person sharing it.

5. Logical architecture

```
Users / Guests
|
v
Cloudflare Edge (WAF / DDoS / coarse IP controls)
|
v
Next.js Web + PWA ---- HTTPS REST /api/v1 ----> NestJS API (stateless)
|
+-------------------------+----------------------+
| | |
Identity & Tenant Domain Modules Platform Modules
Resolution & Policy Planning/Guests Audit/Outbox
Vendors/Finance Notifications
Shagun/Memories Lifecycle/Reports
| | |
+-------------------------+----------------------+
|
+--------------------------+------------------+
| | |
MongoDB Atlas Managed Redis Cloudflare R2
System of record BullMQ / limits Private objects
|
Transactional Outbox
|
Outbox Dispatcher
|
Redis / BullMQ
|
Worker Process
(email, reminders, media, exports, webhooks,
reconciliation, deletion, cleanup)
```

The frontend is not trusted to enforce authorisation. The API is stateless across replicas; session and membership state is resolved through authoritative storage or a deliberately short-lived, invalidatable cache. The worker does not expose public HTTP business endpoints except health/readiness endpoints where operationally required.

6. Deployment topology and runtime processes

Process	Responsibilities	Scaling/constraints
Web	Next.js App Router, PWA shell, UI, client-side validation, API calls	Scale independently; no secret provider credentials in browser
API	NestJS REST, authentication, tenant resolution, policies, domain use cases	Stateless replicas; trusted proxy configuration; bounded DB/Redis pools
Worker	BullMQ consumers, scheduled tasks, media, exports, webhook processing, deletion	Separate concurrency and resource limits per queue
MongoDB Atlas	Authoritative records, transactions, indexes, outbox/inbox	Replica-set/transaction-capable deployment required
Managed Redis	BullMQ transport, distributed counters, short-lived coordination	Private network/TLS/auth; memory and eviction policy reviewed
Cloudflare R2	Private original and derivative image objects	No public bucket access; signed URLs short-lived

For the pilot, use managed split deployment where it materially reduces operational burden. Exact web/API/worker host, region and network topology are not selected in this document. The API origin must not be directly reachable around Cloudflare unless protected by equivalent network controls. Production secrets are injected at runtime through the hosting secret manager.

7. Hosting and environment strategy

- Environments: local development, shared staging, production. Use isolated databases, buckets, Redis instances/namespaces, provider credentials and webhook secrets per environment.
- Never use production guest personal data in local development or screenshots. Use synthetic fixtures in staging.

- Staging uses provider sandbox/test modes and synthetic UPI payment flows; no real funds are collected in automated tests.
- Keep deployment artifacts immutable and configuration externalised.
- Separate API and worker deployments may use the same versioned application image with different commands/entrypoints.
- Select regions based on latency, data handling requirements, provider availability and cost; do not claim India-only residency based on an Asia-Pacific preference.

8. Backend bounded contexts and module ownership

Module	Owns	Must not own
Identity	Users, credentials, account sessions, verification/reset tokens	Wedding domain permissions
Workspace	Wedding, memberships, invitations, membership periods, workspace profile	Guest RSVP business rules
Authorization	Permission catalogue, role grants, policy evaluation, permission versioning	Persistence of unrelated domain aggregates
Governance	Dual-approval requests, proposal snapshots, approval execution	Generic CRUD for all modules
Planning	Functions, publication snapshots, tasks, checklists, dependencies, comments	Guest identity and finance approval
Guests	Households, guest profiles, tags, corrections, merge, household history	Account credentials
RSVP	Function invitations, individual RSVP, additional guest requests	Function publication authority
Vendors	Provider place references, manual vendors, shortlist, enquiries, quotes, availability, booking records	Google Places provider policy itself
Finance	Budgets, allocations, reserve, expenses, approvals, payments, adjustments	Shagun incoming contribution ledger
Shagun	Configuration, payment attempts, contributions, provider events, reconciliation	Holding or custody of funds
Memories	Albums, photo metadata, upload sessions, moderation, reports, removal requests	Binary object storage
Notifications	Notification records/preferences and delivery state	Domain decision-making
Audit	Append-only audit event creation and authorised read/export	Mutating domain state
Platform	Outbox/inbox, jobs, exports, lifecycle deletion, incidents, support access	Owning domain rules

Domain modules expose application use cases and explicit contracts. Avoid circular module dependencies. Shared code belongs in a small kernel containing identifiers, clock, money, error types, tenant context and transaction abstractions—not a generic dumping ground.

9. Repository and codebase structure

```
/
apps/
web/ # Next.js App Router + PWA
api/ # NestJS HTTP bootstrap
worker/ # NestJS application context / BullMQ consumers
packages/
contracts/ # Runtime DTO schemas and API contracts
domain/ # Domain primitives and shared value objects
config/ # Typed environment parsing
database/ # Mongo connection, transaction helpers, migrations
observability/ # Logging, metrics, tracing adapters
eslint-config/
tsconfig/
infra/
environments/
deployment/
monitoring/
docs/
adr/
runbooks/
threat-model/
data-retention/
```

Use runtime validation (for example, Zod or class-validator) at API boundaries. TypeScript types do not validate untrusted JSON. Keep provider SDK imports inside adapter packages/modules. Keep frontend contracts free of server-only secrets and database types.

10. Multi-tenancy model (MT-ADR-01)

10.1 Tenant identity

One wedding workspace is one tenant. The pilot uses one shared MongoDB Atlas database with shared collections and logical isolation. Every tenant-owned record carries weddingId, or is only reachable through a validated tenant-scoped parent relationship. User accounts are global; memberships connect users to weddings.

10.2 Membership cardinality and lifecycle

- At most one membership per user may be ACTIVE or SUSPENDED at a time.
- SUSPENDED membership continues to occupy the membership slot.
- A user may have multiple historical membership periods and may join a new wedding after prior membership becomes ENDED.
- A Wedding B invitation acceptance is rejected with HTTP 409 while the user has ACTIVE or SUSPENDED membership in Wedding A. Do not consume or revoke the Wedding B invitation; do not automatically remove the user from Wedding A.
- Same-wedding rejoin reuses the membership record and appends a new membership-period history record.
- Membership termination is separate from global account deletion. Workspace deletion terminates membership access but does not automatically delete the global account.

10.3 Tenant resolution

Member API flow: authenticate account/session → validate account state → load active membership → resolve weddingId server-side → load permission version/role grants → attach immutable TenantContext to request → execute use case → tenant-scoped repository access. Never treat weddingId from URL, body, query or header as authority. If a route includes a weddingId for resource addressing, compare it to the resolved tenant and reject mismatch.

```
interface TenantContext {
actorType: "MEMBER";
userId: string;
weddingId: string;
membershipId: string;
role: "CO_OWNER" | "ORGANISER" | "TREASURER";
permissionVersion: number;
requestId: string;
}
```

10.4 Guest context

Guest authentication is separate. OTP verification resolves a wedding-scoped guest identity and issues a guest session containing guestId, weddingId, permitted scope, expiry and revocation state. Shared phone numbers do not merge guest identities. Guest sessions cannot cross weddings or act for other guests.

10.5 Repository enforcement

- Repository methods accept TenantContext rather than arbitrary weddingId strings.
- Every find/update/delete/aggregate query includes weddingId scope; relationship lookups validate both parent and child weddingId.
- Use tenant-scoped object key prefixes, Redis namespaces, cache keys, queue payloads, exports, notifications and audit queries.
- Workers validate tenant and resource relationship before acting; queue payload weddingId is not itself authorisation.
- Cross-tenant negative tests are mandatory for every tenant-owned API and asynchronous consumer.

10.6 Membership unique index

```
db.weddingMemberships.createIndex(
{ userId: 1 },
{
unique: true,
partialFilterExpression: {
status: { $in: ["ACTIVE", "SUSPENDED"] }
}
}
);
```

This partial unique index is a final concurrency guard. Invitation acceptance still runs in a MongoDB transaction and handles duplicate-key conflicts as a stable 409 error. The membership record may be reused on same-wedding rejoin; membershipPeriods records prior join/end periods.

11. Authentication and session architecture

- Email/password registration, Google OAuth, email verification, login/logout, password reset and current-session inspection are supported.
- Email verification is required before workspace creation and invitation acceptance.
- Password hashes use Argon2id or bcrypt with reviewed parameters; never store plaintext passwords.
- Use Secure, HttpOnly cookies with appropriate SameSite policy; apply CSRF protection to cookie-authenticated state-changing requests where required.
- Refresh tokens rotate on use; detect reuse and revoke affected token family/session.
- Session revocation must take effect promptly. Do not rely on long-lived JWT role claims as the sole source of current membership or permission.
- Account sessions and wedding membership are separate. Revoking a session does not delete membership; ending membership invalidates access even if a refresh token exists.
- Use generic responses for login/reset/verification where account enumeration is a concern. Store verification/reset tokens hashed and expire them.

12. Guest OTP and guest-session architecture

- OTP challenge is bound to wedding, guest/phone context, purpose, expiry and attempt count.
- Do not reveal private household members before successful OTP verification.
- OTP send and verify endpoints have separate phone/IP/challenge limits and cooldowns.
- OTP is single-use; successful verification issues a random, opaque, revocable guest session.
- Guest session has weddingId and guestId scope, expiry and revocation; every guest API validates it.
- The exact OTP provider, channel, lifetime, delivery policy and guest-session duration remain open decisions.
- Guest session reuse is allowed until expiry/revocation per WF05/WF09; session cannot grant access to other household members.

13. Authorization architecture

13.1 Policy evaluation

Use a central AuthorizationService with domain-specific policy methods. Controllers must not contain ad hoc role-string checks. Each protected use case evaluates principal type, membership/session status, permission, resource scope, resource state and any workflow-specific approval condition. Repositories still enforce weddingId for defence in depth.

13.2 Role and permission outline

Capability	Co-owner	Organiser	Treasurer	Guest
Workspace governance	Both co-owners approve	No	No	No
Functions publish/cancel	Dual approval	No unilateral publish/cancel	No	No
Tasks	All tenant tasks	Permission + scope	No default task access	No
Guest directory	Full permitted	Permission + scope	No default access	Self/eligible invitation only
RSVP assisted update	Yes	Explicit permission + scope	No	Self only
Vendor selection/removal	Yes	Shortlist/enquiry only if permitted	No	No
Finance visibility	Full	Narrow operational own-item only if permitted	Full permitted finance	Own Shagun history only
Normal expense approval	Exception only when Treasurer is creator/beneficiary	No	Yes	No
Treasurer appointment	Both co-owners	No	No	No
Photo moderation	Yes	photos.moderate	No default access	Report/request; upload if invited
Audit log	Both co-owners	No	No	No
Shagun ledger	Full	No	Full	Own contributions only

13.3 Permission versioning

Membership and permission changes increment permissionVersion. Sensitive requests resolve current permissions from authoritative storage or a short-lived cache with explicit invalidation. Never allow stale JWT role claims to restore access after suspension/removal.

14. MongoDB data model and collection ownership

All tenant-owned documents should include weddingId using one consistent BSON type, plus createdAt/updatedAt where applicable. Use schema validation at application boundaries and MongoDB validators for critical shapes where practical. Do not store photo binaries, provider secrets, raw UPI credentials or unnecessary raw webhook payloads in MongoDB.

Collection group	Collections	Tenant scope
Identity	users, userSessions, emailVerificationTokens, passwordResetTokens	Global
Workspace	weddings, weddingMemberships, membershipPeriods, invitations	Wedding-scoped or global identity link
Governance	governanceRequests, governanceApprovalEvents	Wedding
Planning	weddingFunctions, functionPublicationSnapshots, tasks, taskAssignments, taskChecklistItems, taskDependencies, taskComments, taskActivityEvents	Wedding
Guests	households, guests, guestRelationshipTags, guestCorrectionRequests, guestMergeOperations, guestHouseholdHistory	Wedding
RSVP	functionHouseholdInvitations, functionGuestInvitations, rsvps, rsvpEvents, additionalGuestRequests, additionalGuestRequestEvents	Wedding
Vendors	vendorReferences, weddingVendors, vendorServices, vendorEnquiries, vendorEstimates, vendorQuotes, vendorAvailabilityChecks, vendorSelections, vendorBookingRecords	Reference may be global/provider; workspace relationship is wedding-scoped

Collection group	Collections	Tenant scope
Finance	weddingBudgets, budgetCategories, budgetGovernanceRequests, budgetReserveAllocations, expenseEstimates, vendorQuotes, expenses, expenseApprovalEvents, expensePayments, expenseAdjustments, financeAuditEvents	Wedding
Shagun	shagunSettings, shagunConfigurationRequests, shagunPaymentIntents, shagunContributions, shagunProviderEvents, shagunReconciliationEvents, shagunAcknowledgements	Wedding
Memories	photoAlbums, weddingPhotos, photoReports, photoRemovalRequests, photoAuditEvents, uploadSessions	Wedding
Platform	notifications, notificationPreferences, auditEvents, outboxEvents, webhookInbox, exportJobs, securityIncidents, dataAccessRequests, dataCorrectionRequests, supportRecoveryCases, weddingDeletions, weddingDeletionTombstones	Mixed; tenant fields where applicable

Collection names are logical design names, not a mandate to create every collection before its owning feature is implemented. Avoid duplicating the same quote collection under multiple modules; the vendor quote aggregate should have one owner and finance may reference it.

15. Core document shapes (illustrative)

15.1 Membership

```
{
_id, userId, weddingId,
role: "CO_OWNER" | "ORGANISER" | "TREASURER",
status: "ACTIVE" | "SUSPENDED" | "ENDED",
permissionVersion: 1,
joinedAt, endedAt, endReason,
createdAt, updatedAt
}
membershipPeriods: {
_id, membershipId, weddingId, userId,
startedAt, endedAt, endReason, createdBy
}
```

15.2 Guest and RSVP

```
{
_id, weddingId, householdId,
displayName, normalizedName,
phoneRefs: [{ phoneE164, isPrimary }],
relationshipTags: [{ side, relatedPersonId, relationshipType, label }],
status: "ACTIVE" | "ARCHIVED",
version, createdAt, updatedAt
}
rsvps: {
_id, weddingId, functionId, guestId,
response: "NOT_YET_RESPONDED" | "ATTENDING" | "NOT_ATTENDING",
source: "SELF" | "ASSISTED",
actorId, respondedAt, version
}
```

15.3 Expense and payment

```
{
_id, weddingId, title, categoryId,
amountPaise, currency: "INR",
beneficiaryGuestOrMemberId, createdBy,
status, approvalRoute, expectedVersion,
approvedAt, createdAt, updatedAt
}
expensePayments: {
_id, weddingId, expenseId,
amountPaise, currency: "INR",
recordedBy, paidAt, evidenceObjectKey?,
status: "RECORDED" | "REVERSED",
createdAt
}
```

15.4 Photo

```
{
_id, weddingId, functionId, albumId,
uploaderType: "MEMBER" | "GUEST",
uploaderId, objectKeys: {
quarantine?, original?, display?, thumbnail?
},
contentType, sizeBytes, checksum,
moderationStatus, caption?,
version, createdAt, updatedAt
}
```

15.5 Shagun contribution

```
{
_id, weddingId, guestId, paymentIntentId,
amountPaise, currency: "INR",
provider, providerTransactionId,
status, verifiedAt, reversedAt?,
createdAt, updatedAt
}
```

Shapes are illustrative. Final DTOs, field names, encryption/redaction decisions and MongoDB validators must be defined alongside implementation. Avoid embedding unbounded arrays such as all RSVP history or all photo IDs in a single wedding document.

16. Indexing and query strategy

Collection	Index	Purpose
weddingMemberships	unique {userId:1} partial status ACTIVE/SUSPENDED	One active/suspended membership per account
weddingMemberships	unique {weddingId:1,userId:1}	One membership identity per wedding if record is reused
guests	{weddingId:1,householdId:1,status:1}	Household guest listing
guests	{weddingId:1,normalizedName:1}	Duplicate review/search
rsvps	unique {weddingId:1,functionId:1,guestId:1}	One RSVP per guest/function
functionGuestInvitations	unique {weddingId:1,functionId:1,guestId:1}	Prevent duplicate invitations
tasks	{weddingId:1,status:1,deadline:1}	Task dashboards and overdue lists
tasks	{weddingId:1,functionId:1,status:1}	Function task listing
notifications	{weddingId:1,recipientId:1,createdAt:-1}	Recipient notification feed
auditEvents	{weddingId:1,createdAt:-1}	Co-owner audit timeline
outboxEvents	{status:1,nextAttemptAt:1,createdAt:1}	Dispatcher polling
webhookInbox	unique {provider:1,providerEventId:1}	Webhook deduplication
shagunContributions	unique {provider:1,providerTransactionId:1} sparse/partial as appropriate	Prevent duplicate provider transactions
weddingPhotos	{weddingId:1,functionId:1,moderationStatus:1,createdAt:-1}	Gallery and moderation
weddingDeletions	{status:1,nextAttemptAt:1}	Deletion worker scheduling

Index design must be validated against actual query patterns with explain('executionStats'). Add only indexes justified by read/write patterns; every additional index increases write and storage cost. All tenant-scoped query indexes should include weddingId early where appropriate.

17. Transaction boundaries and consistency

- Create workspace + first co-owner membership + audit/outbox event in one MongoDB transaction.
- Accept co-owner/organiser invitation + membership change + invitation state + workspace update + audit event in one transaction.
- Governance second approval + final action execution + approval event + audit/outbox in one transaction.
- Publish/cancel function: validate both approvals, write immutable publication/cancellation snapshot and audit/outbox atomically.
- RSVP write + RSVP event + notification outbox record in one transaction where cross-record consistency is required.
- Expense approval + approval event + budget/commitment projection update (if persisted) + audit/outbox in one transaction.
- Payment recording/correction uses append-only payment/adjustment records; never silently overwrite prior financial events.
- Webhook: persist authenticated/minimised inbox record before acknowledging; worker applies event idempotently in a separate transaction.
- Photo completion: confirm object metadata, create processing job/outbox; status changes only after worker validation.
- Deletion: each deletion step is idempotent and checkpointed; never rely on one giant transaction for large tenant deletion.

MongoDB transactions require a transaction-capable Atlas deployment. Keep transactions short; do not call external providers while holding a MongoDB transaction open. Use optimistic concurrency with version/expectedVersion for editable aggregates. Use unique indexes as final concurrency guards.

18. API conventions and contract

- REST base path: /api/v1. JSON request/response. UTC ISO-8601 timestamps. Monetary values use amountPaise and currency='INR'.
- Runtime-validate every path parameter, query and body. Reject unknown privileged fields; use DTO allowlists to prevent mass assignment.
- Use explicit command routes for state transitions, e.g. POST /expenses/{id}/approve, not PATCH {status:'APPROVED'}.
- List endpoints use cursor pagination, bounded limit (recommended maximum 100), allowlisted filters and allowlisted sort keys.
- Use Idempotency-Key for retry-sensitive commands; bind key to actor, route and canonical request hash.

- Use expectedVersion for optimistic concurrency; return 409 on stale writes.
- Stable error codes include VALIDATION_ERROR, UNAUTHENTICATED, FORBIDDEN, NOT_FOUND, CONFLICT, RATE_LIMITED, PROVIDER_UNAVAILABLE and ACTIVE_WEDDING_MEMBERSHIP_EXISTS.

```
Success:
{
"data": { "id": "resource-id", "version": 4 },
"meta": { "requestId": "req_..." }
}

Error:
{
"error": {
"code": "ACTIVE_WEDDING_MEMBERSHIP_EXISTS",
"message": "This account cannot accept another wedding invitation yet."
},
"meta": { "requestId": "req_..." }
}
```

19. API surface by domain

Domain	Representative endpoints	Notes
Identity	POST /auth/register; POST /auth/verify-email; POST /auth/login; POST /auth/refresh; POST /auth/logout; POST /auth/password-reset	Secure cookie/session flows; rate-limited
Workspace	POST /weddings; GET /weddings/current; GET /weddings/{id}; PATCH /weddings/current/profile	Current tenant derived from membership; ID path never grants access
Invitations	POST /weddings/current/invitations/co-owner; POST /invitations/{token}/accept; POST /weddings/current/invitations/organisers	Acceptance transactional; opaque token; governance gates
Governance	POST /governance-requests; POST /governance-requests/{id}/approve; POST /governance-requests/{id}/reject	Immutable proposal snapshot and expectedVersion
Functions	POST /functions; POST /functions/{id}/publish-request; POST /functions/{id}/complete; POST /functions/{id}/cancel-request	No generic status patch
Tasks	GET/POST /tasks; PATCH /tasks/{id}; POST /tasks/{id}/complete; POST /tasks/{id}/reopen; POST /tasks/{id}/assign	Permission and scope checks
Guests	GET/POST /households; GET/POST /guests; POST /guests/{id}/corrections; POST /guest-merges	Review/dual approval flows
Guest RSVP	POST /guest-auth/otp; POST /guest-auth/verify; GET /guest/functions/{functionId}; PUT /guest/functions/{functionId}/rsvp	Guest principal only; self response
Vendors	GET /vendor-discovery; POST /vendors/manual; POST /vendors/{id}/shortlist; POST /vendors/{id}/quotes	Places adapter; selection rules
Finance	GET /finance/dashboard; POST /expenses; POST /expenses/{id}/approve; POST /expenses/{id}/payments; POST /budget-governance-requests	Treasurer and dual approval routes
Shagun	POST /shagun/configuration-requests; POST /shagun/payment-intents; GET /shagun/my-contributions; POST /webhooks/{provider}/payments	Webhook route validates provider signature; no client success endpoint
Memories	POST /photos/upload-sessions; POST /photos/{id}/complete; POST /photos/{id}/moderate; GET /photos; POST /photos/{id}/removal-requests	Signed URLs issued after authorisation
Notifications	GET /notifications; POST /notifications/{id}/read; PATCH /notification-preferences	Recipient scoped
Audit	GET /audit-events; POST /audit-exports	Co-owner only; export audited
Lifecycle	POST /workspace-cancellation-requests; POST /workspace-date-change-requests; POST /workspace-exports	Deletion execution is internal worker-only

20. Critical workflow sequences

20.1 Invitation acceptance and one-membership rule

```
Invitee -> authenticate + verify invited email
-> load invitation and validate token/expiry/email
-> transaction: query ACTIVE/SUSPENDED membership
-> if another wedding membership exists:
return 409 ACTIVE_WEDDING_MEMBERSHIP_EXISTS
do not consume/revoke invitation
-> otherwise create/reactivate membership + membership period
-> mark invitation accepted + audit/outbox
-> commit
-> duplicate-key race is mapped to same 409
```

20.2 Function publication

```
Co-owner requests publication
-> immutable proposal/snapshot created
-> first co-owner approval recorded
-> second distinct co-owner approval
-> transaction verifies proposal version and function state
-> publish immutable snapshot + update state + audit/outbox
-> guest pages read only current published snapshot
```

20.3 Guest RSVP

```
Guest opens function invitation page
-> submit phone/identity hint
-> API creates wedding-scoped OTP challenge
-> provider sends OTP
-> verify OTP (attempt/expiry checks)
-> issue scoped guest session
-> load only invited eligible guest records
-> guest submits self RSVP
-> transaction validates deadline + invitation + guest identity
-> update RSVP/version + append RSVP event + outbox
-> return confirmed state
```

20.4 Expense approval and payment

```
Creator submits expense
-> determine approval route
-> normal expense: Treasurer; Treasurer-created/beneficiary: both co-owners
-> creator self-approval rejected
-> transaction records approval + approved commitment
-> Treasurer records one or more payment entries
-> outstanding = approved commitment - valid payments
-> correction/void uses dual approval and append-only adjustment
```

20.5 Digital Shagun

```
Verified guest -> API creates payment intent with idempotency key
-> provider adapter creates UPI QR/app link
-> guest completes payment outside platform
-> provider webhook arrives
-> verify signature, merchant, amount, currency, reference and timestamp/replay
-> persist minimal webhook inbox event uniquely
-> acknowledge provider
-> worker fetches provider status if required
-> transaction updates intent/contribution idempotently
-> issue acknowledgement only after verified success
```

20.6 Guest photo upload

```
Guest session -> verify individual function invitation and upload eligibility
-> API creates upload session and short-lived presigned PUT to quarantine key
-> browser uploads directly to private R2
-> completion API HEAD/checks object, size and checksum
-> outbox queues image processing
-> worker decodes/validates/scans where feasible, strips GPS EXIF, creates derivatives
-> guest photo = PENDING_REVIEW; authorised member photo = APPROVED after technical validation
-> authorised GET URL generated only after access and status checks
```

20.7 Workspace deletion

```
Lifecycle trigger -> create deletion manifest/job
-> mark tenant inaccessible and freeze writes
-> revoke member/guest sessions and invitations
-> stop reminders and recurring jobs
-> delete R2 originals/derivatives/temp objects
-> delete tenant domain collections in bounded batches
-> remove caches/queue artefacts and request provider cleanup
-> verify each step and record checkpoints
-> retain minimal tombstone
-> mark COMPLETED only after required steps verified
```

21. State machines and invariants

Aggregate	States / transitions	Invariant
Membership	INVITED → ACTIVE; ACTIVE ↔ SUSPENDED; ACTIVE/SUSPENDED → ENDED	One ACTIVE/SUSPENDED membership per user
Invitation	PENDING → ACCEPTED/EXPIRED/REVOKED/SUPERSEDED	Single use; exact verified email; expiry
Function	DRAFT → PUBLISHED → COMPLETED; PUBLISHED → CANCELLED	Publish/cancel dual co-owner approval; published snapshot immutable
Task	TO_DO/IN_PROGRESS/BLOCKED/COMPLETED	Mandatory checklist and dependencies required to complete
RSVP	NOT_YET_RESPONDED ↔ ATTENDING/NOT_ATTENDING until deadline	Unique per wedding/function/guest; self only

Aggregate	States / transitions	Invariant
Expense	DRAFT → PENDING_APPROVAL → APPROVED/REJECTED → PAYMENT_IN_PROGRESS/SETTLED; VOID_PENDING_APPROVAL → VOIDED	Approval route and creator separation; payments separate
Photo	PENDING_REVIEW → APPROVED/REJECTED; APPROVED ↔ HIDDEN; → REMOVED	Guest uploads moderated; permanent removal co-owner decision
Shagun	CREATED → PENDING_PROVIDER → PENDING_VERIFICATION → SUCCESSFUL/FAILED; reversal/refund separately recorded	Only trusted provider evidence can confirm success
Workspace	SETUP_IN_PROGRESS → TWO_CO_OWNERS_ACTIVE → POST_WEDDING → deletion; cancellation → inaccessible + deletion	Cancellation irreversible; deletion retryable and tombstoned

All transitions validate current state, actor policy, preconditions and expectedVersion. Never expose generic client-controlled status mutation for privileged states.

22. Financial model and calculations

- Store all monetary amounts as integer paise; do not use floating-point arithmetic for money.
- Budget total = sum(category allocations) + reserve. Reserve is included in total budget.
- Pending expense exposure is separate from approved commitments.
- Approved commitments exclude rejected, cancelled and voided expenses.
- Outstanding commitments = approved commitments – valid payment entries, subject to explicit adjustment/reversal handling.
- Reserve remaining = reserve allocation – approved reserve usage.
- Shagun contributions are incoming funds and are not netted against expense or budget figures.
- Payment entries are append-only; corrections are represented by explicit adjustments/reversals.
- Enforce payment amount not exceeding the approved expense amount unless a formally approved amount change has occurred.

23. Digital Shagun provider integration

- Define a ShagunProvider interface: createPaymentIntent, fetchPaymentStatus, verifyWebhook, parseProviderEvent, fetchReversalStatus where supported.
- Do not select a provider until it can demonstrate the actual funds flow to the couple's beneficiary, server-side verification, signed webhook or authenticated status API, reconciliation and reversal/refund handling.
- Validate provider merchant/account, internal intent reference, provider transaction ID, amount, currency, beneficiary mapping and event authenticity.
- Persist provider event ID uniquely; deduplicate retries. Do not trust browser redirect or guest reference.
- Webhook acknowledgement occurs only after durable inbox persistence. If MongoDB is unavailable, do not return success acknowledgement that would lose the event; rely on provider retry contract.
- Reconciliation worker checks pending/ambiguous intents with provider API. Treasurer may initiate reconciliation but cannot manually mark success.
- Store only necessary references and redacted metadata. Never collect UPI PIN, payment OTP or bank password.

24. Photo and object-storage architecture

- Use private Cloudflare R2 bucket; MongoDB stores metadata and opaque keys, not binary content.
- Browser uploads directly using short-lived presigned PUT URLs generated by NestJS after tenant, function invitation and quota checks.
- Upload to quarantine/pending prefix. Completion endpoint checks object existence, size and checksum; client assertion alone is insufficient.
- Worker validates actual content and decode, dimensions, file size, permitted format, removes GPS/unnecessary EXIF and generates display/thumbnail derivatives.
- Member upload auto-approves only after technical validation; guest upload remains PENDING_REVIEW.
- Signed GET URL is issued only after current tenant/session, permission and moderation status checks; short TTL recommended, e.g. five minutes, subject to UX review.
- Do not store permanent public URLs. Do not cache protected originals/signed URLs offline in PWA.
- R2 Asia-Pacific placement is a preference, not an India data-residency guarantee. Verify placement and legal needs before production.

25. Background jobs, outbox and inbox

25.1 Transactional outbox

When a domain transaction must trigger asynchronous work, write the domain change and outbox event in the same MongoDB transaction. A dispatcher claims pending outbox records and publishes to BullMQ. If Redis is unavailable, outbox records remain durable and are retried after

recovery.

25.2 Job envelope

```
{
eventId, jobType, schemaVersion,
weddingId?, actorId?,
aggregateType, aggregateId, aggregateVersion,
correlationId, idempotencyKey,
createdAt, payload
}
```

25.3 Worker rules

- Assume at-least-once delivery; handler must be idempotent.
- Use bounded retries with exponential backoff and jitter; exhausted jobs enter failed/dead-letter review.
- Use deterministic job IDs where appropriate and distributed locks for scheduled work.
- Revalidate tenant, aggregate state and permissions where job semantics require it; do not trust stale payload state.
- Separate queue concurrency for email, media processing, exports, webhooks and deletion so one workload cannot starve critical work.

25.4 Webhook inbox

Persist provider webhook event with unique providerEventId and minimal redacted payload before acknowledging. Process asynchronously and idempotently. Retain enough evidence for reconciliation and support without retaining unnecessary sensitive payloads.

26. Redis, caching and rate limiting

26.1 Redis responsibilities

- BullMQ queue transport and job coordination.
- Distributed rate-limit counters and short-lived OTP/session coordination where appropriate.
- Short-lived coordination locks and ephemeral data only when justified.
- No authoritative user, membership, permission, RSVP, finance or payment state.

26.2 Rate-limit layers

Layer	Responsibility
Cloudflare	DDoS/WAF/bot controls and coarse IP rules; edge layer does not replace identity-aware controls
NestJS	Route-specific limits keyed by account, user, guest, phone, session, wedding and IP as appropriate
Managed Redis	Atomic distributed counters with TTL shared across API replicas
Provider budget	Separate daily/monthly email and OTP quota controls
Upload quotas	File size, batch size, session, wedding storage and processing concurrency

26.3 Initial strict pilot limits

Endpoint	Initial limit	Keys
Login	5 failed / 15 min	Account + IP
Registration	3 / hour	IP
Password reset	3 / hour	Account
Verification email	3 / hour	Account
OTP send	3 / 10 min and 5 / hour	Phone + IP
OTP verify	5 / challenge	Challenge + IP
Invitation acceptance	5 / 15 min	IP
Guest RSVP	10 / 10 min	Guest + session
Photo upload initiation	5 / 10 min	User/guest
Upload completion	10 / 10 min	User/guest
Photo reports	5 / hour	Guest
Authenticated API	60 / min	User
Public reads	30 / min	IP, with route exceptions

These are initial defaults, not immutable product limits. Tune with load tests and false-positive monitoring. Shared family Wi-Fi and mobile carrier NAT require care; do not rely on IP-only blocks. Use atomic Redis operations (Lua or equivalent) and return 429 with Retry-After. Login should use progressive cooldown rather than permanent account lockout.

26.4 Redis outage policy

Hybrid policy: fail closed for authentication, OTP, email-triggering and upload-sensitive operations. Ordinary reads may use conservative per-instance emergency limits, acknowledging that local counters are weaker and non-distributed. Outbox remains in MongoDB while Redis is unavailable. Do not silently disable all protection.

27. Email and notification architecture

- Wedding operational notifications are in-app only. Email is reserved for account/security transactions: verification, reset, invitations and account/security changes.
- Brevo is the recommended initial transactional email provider; exact paid tier, quota and pricing must be verified. A target near 300/day has no headroom on a 300/day free tier.
- Use provider adapter interface and server-side API key. Authenticate delivery/bounce webhooks and update delivery state idempotently.
- Provider accepted ≠ delivered. Track QUEUED, PROCESSING, ACCEPTED, DELIVERED, BOUNCED, FAILED and SUPPRESSED as distinct operational states.
- Use MongoDB outbox + BullMQ, separate priority queues or priority policy, global daily/monthly budget, per-account limits and provider throughput controls.
- Do not retry ambiguous timeout blindly across providers; reconcile first to avoid duplicate messages.
- In-app notifications are generated only from committed domain events, idempotently; deep links recheck permission.

28. Vendor discovery adapter

- Use official Google Places API through a VendorDiscoveryProvider interface.
- Keep provider place reference distinct from wedding-specific vendor shortlist and quote records.
- Apply field masks, attribution and caching/retention restrictions from current provider terms.
- Manual vendor records are supported and visibly marked manual.
- Availability is manually recorded with explicit state and confirmation date; never inferred from opening hours or a provider listing.
- A selected vendor is not proof of booking, contract, availability or payment.

29. Audit, privacy and support access

- Audit events include weddingId, actor type/id, action, resource type/id, outcome, reason, redacted changes, requestId/correlationId and timestamp.
- Audit events are application-level append-only and visible only to both co-owners. Sensitive reads and exports are recorded where appropriate.
- Never log credentials, OTPs, raw tokens, payment secrets or full webhook payloads.
- Application append-only is not tamper-proof against privileged DB operators; optional hash chaining/signed checkpoints and protected backups are stronger assurance options.
- Support access is break-glass: scoped wedding, reason/ticket, approval, expiry, read-only by default, every action audited and access revoked at expiry.
- Infrastructure/database operators may have technical privilege; control with least-privilege IAM, just-in-time access where available, secret rotation and access logs.

30. Workspace lifecycle and deletion orchestration

- Wedding date is date-only with Asia/Kolkata semantics. POST_WEDDING begins after wedding date.
- Deletion due at 00:00 IST on day 31 after wedding date. Convert calculated instant to UTC for scheduling; store original wedding-local date and calculated deadline.
- Date change after original date has passed requires both co-owners' approval and is allowed only before deletion begins. Worker rechecks current approved date before execution.
- Cancellation requires both co-owners; second approval is final. Immediately revoke access, guest sessions, invitations, public pages, RSVP and upload access.
- Only minimal generic cancellation notice remains visible for seven days; private workspace data is inaccessible immediately.
- Deletion manifest has per-step status and checkpoint. States: QUEUED, IN_PROGRESS, RETRY_PENDING, BLOCKED, COMPLETED, FAILED_REQUIRES_REVIEW.
- Steps: freeze writes; revoke access; stop scheduled jobs; delete object storage originals/derivatives/temp uploads; delete tenant records and derived data; request provider cleanup; verify; tombstone.
- Keep tenant inaccessible during retry/failure. Do not mark complete until required cleanup is verified or exception formally documented.
- Backup target proposed maximum 30 days subject to provider/legal validation. Replay tombstones before restored environment is made live.
- Final export can be requested before cancellation's second approval; export cannot delay cancellation or deletion.

31. Security threat model

Threat	Control
Tenant escape / IDOR	Server-resolved tenant, scoped repositories, same-tenant relationship checks, negative tests
Membership race	Unique partial index, transaction, duplicate-key-to-409 mapping
Privilege escalation	Central policy service, permission version, no client role claims
OTP brute force / enumeration	Challenge limits, phone/IP controls, generic responses, no household reveal pre-verification
Session theft/replay	HttpOnly Secure cookies, refresh rotation, revocation, CSRF controls
Webhook spoof/replay	Signature validation, timestamp/replay policy, unique event ID, provider status reconciliation
Duplicate payment	Idempotency key, unique provider transaction, amount/reference validation
Malicious upload	Quarantine, content decode validation, size/quota, scan where feasible, EXIF stripping
XSS / injection / mass assignment	Output encoding, runtime validation, DTO allowlists, safe query construction
Sensitive data leakage	Private storage, signed URLs, redacted logs, scoped exports, no public guest/finance data
Abuse / cost exhaustion	Cloudflare, NestJS/Redis limits, provider budgets, storage quotas
Deletion resurrection	Tombstones replayed on restore, inaccessible until reconciliation

Security review must include web application risks, API authorisation, dependency/supply-chain scanning, secrets management, provider webhook verification, privacy review and adversarial tenant isolation tests.

32. Observability and operational readiness

- Structured JSON logs include requestId, correlationId, route, status, latency, stable error code, actorId/weddingId only where safe; redact personal data and secrets.
- Metrics: API latency/error rate, DB pool saturation/slow queries, Redis latency/memory/connections, queue depth/oldest age, failed jobs, webhook lag, email bounces, OTP spend, photo processing failures, deletion backlog and membership conflict rate.
- Liveness indicates process health; readiness indicates ability to serve requests given critical dependencies. Avoid restart loops for transient optional-provider failures.
- Set alert thresholds after baseline measurement. Initial alert categories: elevated 5xx, DB saturation, Redis outage, queue age growth, webhook backlog, provider quota thresholds, suspicious auth spikes and deletion failures.
- Maintain incident severity SEV_1–SEV_4 and incident lifecycle DETECTED, TRIAGED, CONTAINMENT, INVESTIGATION, RESOLUTION, CLOSED.

33. Operational runbooks

Incident	Response
MongoDB outage	Fail closed for writes and sensitive reads. Do not accept client claims of payment success. Restore service and reconcile provider inbox/outbox.
Redis outage	Apply hybrid fail policy; outbox accumulates in MongoDB; restore Redis and republish pending events.
R2 outage	Block new upload completion and signed download issuance gracefully; never mark upload complete until verified.
Worker crash	BullMQ retry with backoff; inspect oldest job age; replay idempotently; alert on sustained backlog.
Email quota exhausted	Defer non-critical mail, preserve outbox, alert; never silently drop security messages; investigate provider quota.
Webhook backlog	Inspect durable inbox, verify signature records and event IDs; replay idempotently; reconcile uncertain payments via provider API.
Suspected account compromise	Revoke sessions, suspend affected non-co-owner membership where authorised, preserve audit evidence, notify relevant co-owner.
Cross-tenant exposure	Contain affected route, revoke access/URLs, preserve logs, investigate scope, notify through approved incident process.
Deletion failure	Keep workspace inaccessible; retry bounded steps; escalate BLOCKED/FAILED_REQUIRES_REVIEW; do not mark deleted.
Support access misuse	Immediately revoke break-glass grant, preserve audit trail, investigate operator and data access.

34. Testing strategy

- Unit: domain invariants, policy evaluation, state transitions, membership lifecycle, money arithmetic, budget calculations, approval routes and idempotency.

- Integration: MongoDB indexes/transactions, repository tenant scoping, outbox/inbox, BullMQ retries, Redis atomic counters and provider adapters with test doubles.
- E2E: registration/verification, workspace onboarding, second co-owner, organiser governance, tasks, guest OTP/RSVP, additional guest requests, finance, vendor shortlist, Shagun webhook, photo moderation and deletion.
- Security: IDOR and cross-tenant access, guest enumeration, membership race, stale permission, webhook replay, duplicate provider transaction, signed URL misuse, mass assignment and support privilege.
- Concurrency: two simultaneous invite acceptances, dual approval races, duplicate RSVP, repeated payment webhook, simultaneous expense approval and deletion worker re-entry.
- Load: RSVP burst near deadline, upload bursts, dashboard queries, webhook retries and deletion batches. Do not invent scale targets; profile the pilot workload.

35. CI/CD, migration and release strategy

- CI runs formatting/lint, type-check, unit/integration tests, build, dependency and secret scanning.
- Deploy immutable artifacts. Run database migrations as an explicit controlled step with backup/rollback plan.
- Use expand-migrate-contract: deploy backward-compatible schema, backfill, switch reads/writes, then remove old shape in a later release.
- Deploy staging first with synthetic data and provider sandbox; run smoke, security and critical workflow tests before production promotion.
- Use distinct environment credentials and least privilege. Rotate provider and webhook secrets using documented procedures.
- Rollback application independently where schema remains backward-compatible; destructive data migration rollback requires a separately tested recovery plan.

36. Backup, restore and disaster recovery

- Define RPO and RTO before production; do not claim a recovery guarantee until hosting/provider plans support it.
- Enable MongoDB managed backups and point-in-time recovery where available and affordable.
- Object storage must have a separate backup/versioning/recovery plan; database backup alone does not restore photo binaries.
- Test restore into an isolated environment and verify indexes, tenant records, object references, outbox/inbox, deletion tombstones and application compatibility.
- Replay deletion tombstones before restored tenant data becomes accessible.
- Redis can be rebuilt for rate limits and queue transport; durable outbox/inbox and domain records remain in MongoDB.
- Document credential rotation, provider webhook secret rotation, recovery owner and restore rehearsal frequency.

37. Migration path to isolated tenants

- Do not implement database-per-wedding for the pilot. Shared collections are the approved initial model.
- Keep tenant repository interfaces and tenant-aware data access contracts so physical storage can change without rewriting domain rules.
- If future isolation is justified, introduce a tenant storage registry mapping weddingId to storage location and migration state.
- Migration state may be SHARED, MIGRATING, ISOLATED. Define source/target consistency, write freeze or dual-write protocol, copy verification, index recreation, cutover and rollback before executing.
- Do not introduce a routing abstraction across all repositories until a real migration requirement exists.

38. Architecture decision records

ADR	Decision	Status
ADR-001	Modular monolith for pilot; no microservices absent measured need	Accepted
ADR-002	Separate Next.js frontend and NestJS API	Accepted
ADR-003	Separate API and worker processes from shared backend codebase	Accepted
ADR-004	MongoDB Atlas authoritative system of record	Accepted
ADR-005	Managed Redis + BullMQ; MongoDB outbox is durable source	Accepted direction
ADR-006	Shared-database logical multi-tenancy with weddingId and server-resolved context	Accepted
ADR-007	One ACTIVE/SUSPENDED membership per user; no automatic wedding transfer	Accepted
ADR-008	Separate wedding-scoped guest identity/session	Accepted
ADR-009	Transactional outbox/inbox and idempotent consumers	Mandatory
ADR-010	Private Cloudflare R2, presigned uploads and authorised short-lived downloads	Selected direction; residency validation open

ADR	Decision	Status
ADR-011	Cloudflare edge + Redis-backed NestJS rate limiting	Accepted direction
ADR-012	Brevo recommended for transactional email; plan/pricing to verify	Recommendation
ADR-013	Provider-neutral OTP and UPI payment adapters	Mandatory until provider selection
ADR-014	Temporary, scoped, audited break-glass platform support	Accepted
ADR-015	Post-wedding deletion at 00:00 IST day 31; cancellation irreversible	Accepted

39. Open decisions and production blockers

- Hosting provider, deployment region, private origin connectivity and Cloudflare plan/features.
- MongoDB Atlas region/tier, transaction support, backup/PITR plan and connection limits.
- Managed Redis provider, region, memory, eviction, HA and BullMQ recovery terms.
- OTP provider/channel, India delivery routes, cost, expiry, resend, sender/template approval and guest-session duration.
- UPI payment provider, beneficiary onboarding, settlement, fees, webhook contract, status API, refunds/reversals, disputes and legal obligations.
- Brevo paid tier, exact quota and pricing; target volume is approximately 300 emails/day.
- R2 actual placement/residency, account settings, lifecycle/backup and cost.
- Google Places current field masks, attribution, caching and retention terms.
- Reserve approval threshold; photo formats, file/batch/storage quotas and worker concurrency.
- Support recovery proof requirements; privacy notice, consent, retention and applicable Indian legal review.
- SLOs, RPO/RTO, alert thresholds, support response targets and pilot capacity/cost budget.

40. Implementation readiness checklist

- PRD v2.0 approved as product baseline and traceability established for WF01–WF12.
- Tenant context, membership unique index and cross-tenant negative tests implemented before feature rollout.
- MongoDB transaction-capable environment and index migration process verified.
- Central authorization service and permission matrix implemented; stale permission/session invalidation tested.
- Outbox/inbox, worker idempotency, retry and replay patterns tested.
- Provider adapters and sandbox contracts defined before real provider integration.
- UPI provider and beneficiary model validated before Shagun production rollout.
- OTP provider and guest session policies validated before guest-facing launch.
- R2 private bucket, upload quarantine, EXIF processing, signed URL authorisation and deletion tested.
- Cloudflare origin protection, Redis failure policy and rate-limit monitoring tested.
- Audit export, break-glass access, incident response and workspace deletion rehearsed.
- Backup restore, tombstone replay and external provider cleanup rehearsed in staging.
- Legal/privacy/payment review, SLO/RPO/RTO and pilot budget approved.

Appendix A. Key engineering conventions

- Use ObjectId or a consistent opaque identifier strategy; never expose sequential IDs as security controls.
- Use UTC for instants and explicit Asia/Kolkata semantics for wedding-local dates/times.
- Use cursor pagination and bounded query limits; avoid unbounded skip on growing collections.
- Use expectedVersion for concurrent aggregate updates and stable conflict errors.
- Keep public invitation pages to intentionally published information only.
- Avoid storing denormalised data unless update consistency and rebuild strategy are documented.
- Use explicit domain events for state transitions; do not treat audit events as the event-sourcing source of truth.

Appendix B. Traceability from PRD workflows

PRD workflow	Primary architecture modules	Critical controls
WF01 Onboarding	Identity, Workspace, Invitations	Email verification, one membership, transactional acceptance

PRD workflow	Primary architecture modules	Critical controls
WF02 Governance	Governance, Authorization, Workspace	Dual approval, immutable proposal, role/permission version
WF03 Functions/tasks	Planning	Published snapshot, dependency validation, task state machine
WF04 Guest directory	Guests	Household invariants, correction review, dual-approved merge
WF05 RSVP	RSVP, Guest Auth	Self-only, deadline, unique RSVP, scoped guest session
WF06 Budget/expenses	Finance	Paise, approval route, Treasurer exception, append-only payments
WF07 Vendors	Vendors	Places terms, manual source, availability not inferred
WF08 Shagun	Shagun, Webhook Inbox	Provider verification, idempotency, reconciliation
WF09 Photos	Memories, R2, Worker	Private objects, moderation, EXIF removal, scoped URLs
WF10 Dashboards/notifications	Read models, Notifications	Role-specific visibility, idempotent notification events
WF11 Audit/security	Audit, Platform Security	Co-owner-only audit, break-glass, incident records
WF12 Lifecycle/deletion	Lifecycle, Worker, Storage	Dual approval cancellation, deletion manifest, tombstones